Supervised Machine Learning Approach for WSD in Spoken News

Sree Saicharan Vadapalli
ET23BTIT133

Architecture and Methodology

1 Overview

News transcripts are full of words like *bank*, *interest*, *plant* and *run* whose meaning depends on the sentence around them. This project treats word sense disambiguation (WSD) as a supervised classification problem. For every target word I train a separate classifier (a "word expert") that maps the context of an occurrence to a WordNet synset. Two classifiers are compared, a linear SVM and multinomial Naive Bayes, each with two context window sizes, $W = \pm 2$ and $W = \pm 5$.

The pipeline has three stages, one per source file:

File	Responsibility
`src/preprocessing.py`	SemCor extraction, POS tags, lemmas, stopwords
`src/models.py`	Feature extraction and the two classifiers
`src/pipeline.py`	Cross-validated training and testing, metrics

2 Dataset

- **Source**: SemCor, read through NLTK. Its labels are WordNet synsets and it covers the words named in the problem. NLTK's Senseval data only has *hard*, *interest*, *line* and *serve*, labelled with its own sense inventory.
- **Target words**: *bank*, *interest*, *plant* and *run* from the problem statement, plus *state*, *case*, *rate* and *power*, which are common in news, have several senses and have enough tagged examples to train on.
- **Extraction**: every sense-tagged chunk whose WordNet lemma is a target is kept along with its synset, token position and sentence. The sentence is then POS-tagged with NLTK's perceptron tagger. This gives 1,177 labelled occurrences, saved to `data/annotated/semcor_wsd.csv`.
- **Filtering**: senses with fewer than 5 examples are dropped since they cannot appear in every fold of a 5-fold split, leaving 1,092 occurrences.
- **Caveat**: SemCor is written text from the Brown corpus, not speech. Raw transcripts have no punctuation, so the punctuation tags that show up in the POS window here would be missing, and the POS tagger itself was trained on punctuated text and will be less accurate on them.

Word	Occurrences	Senses	Majority sense share
bank	34	2	0.59
case	133	5	0.48
interest	140	7	0.41
plant	107	3	0.59
power	99	6	0.57
rate	116	3	0.59
run	185	12	0.36
state	278	6	0.39

3 Feature vectors

Take a sentence $w_1, \ldots, w_n$ with POS tags $p_1, \ldots, p_n$ and a target word at position t. For a window size W the feature vector has three parts:

1. **Target POS**: the tag p_t, one-hot encoded. This alone separates noun and verb senses such as *plant.n.01* and *plant.v.01*.
2. **Neighbouring POS**: the tags p_{t+k} for $k \in \{-W, \ldots, -1, 1, \ldots, W\}$, each as its own positional feature, so `pos_-1=JJ` and `pos_+1=JJ` are different features.
3. **Window bag-of-words**: counts of the lemmas of w_{t+k} over the same offsets. Tokens are lowercased, stopwords and non-alphabetic tokens are removed, and the rest are lemmatized with the WordNet lemmatizer using the token's POS tag.

Putting it together:

$$\mathbf{x}_t = \text{onehot}(p_t) \;\oplus \bigoplus_{0<|k|\le W} \text{onehot}_k(p_{t+k}) \;\oplus \sum_{0<|k|\le W} \text{onehot}\big(\text{lemma}(w_{t+k})\big)$$

where $\oplus$ is concatenation.

The window is counted on raw token positions, so W means the same thing for the POS and bag-of-words parts. A side effect is that at $W = \pm 2$ the bag-of-words often holds only one or two words, since stopwords take up some of the slots.

A real example from the dataset, with *interest* as the target:

> ... considering the widespread **interest** in the election , the number of voters ...

At $W = \pm 2$ this becomes

```
{pos_0=NN, pos_-2=DT, pos_-1=JJ, pos_+1=IN, pos_+2=DT, bow_widespread=1}
```

At $W = \pm 5$ it also picks up `bow_consider`, `bow_election` and the tags out to `pos_-5` and `pos_+5`.

The feature dicts are turned into vectors with scikit-learn's `DictVectorizer`. It sits inside the model pipeline, so the vocabulary is learned from the training folds only.

4 Classifiers

Both classifiers use scikit-learn defaults; I did no hyperparameter tuning.

Multinomial Naive Bayes picks the sense s that maximizes

$$\hat{s} = \arg\max_s \; \log P(s) + \sum_j x_j \log P(f_j \mid s),$$

where $P(f_j \mid s)$ is estimated from feature counts with Laplace smoothing ($\alpha = 1$). It assumes features are independent given the sense, which is clearly not true for overlapping POS and word features, but it is fast and works well on small data.

Linear SVM (`SVC` with a linear kernel, $C = 1$) learns a separating hyperplane $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ by minimizing

$$\frac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \max\left(0,\, 1 - y_i f(\mathbf{x}_i)\right).$$

Multiple senses are handled one-vs-one, which is what `SVC` does by default. A linear kernel is the usual choice for sparse, high-dimensional text features.

5 Evaluation protocol

With 34 to 278 examples per word, a single train/test split would leave some test sets with fewer than 10 examples. Instead I use stratified 5-fold cross-validation (shuffled, fixed seed) through `cross_val_predict`. Every occurrence is predicted exactly once, by a model that never saw it in training, and the metrics are computed on those predictions.

I report accuracy and macro-averaged precision, recall and F1:

$$\mathrm{P}_s = \frac{TP_s}{TP_s + FP_s}, \quad \mathrm{R}_s = \frac{TP_s}{TP_s + FN_s}, \quad \mathrm{F1}_s = \frac{2\,\mathrm{P}_s\mathrm{R}_s}{\mathrm{P}_s + \mathrm{R}_s}, \quad \text{Macro F1} = \frac{1}{|S|}\sum_{s \in S} \mathrm{F1}_s.$$

Sense distributions are skewed, so accuracy alone rewards a model that just predicts the most common sense. Macro F1 weighs every sense equally and shows whether rare senses are actually being learned.